

AMC PROJECT 1

BISECTION METHOD SOLVER

I. Project Objective

The primary objective of this project is to develop a Python program that accurately solves non-linear equations using the Bisection Method. The script is designed to find the root of a function by repeatedly narrowing down an interval where a sign change occurs. It generates a detailed, 6-column iteration table that allows the user to track the boundaries, the midpoint, and the narrowing bracket width (error) until mathematical convergence is achieved.

II. Algorithm Documentation

```
START
    INITIALIZE application window (GUI)
    WAIT for user click
    GET user inputs (requires left and right bounds)
    VALIDATE bounds trap the root
    BUILD mathematical function
    RUN Bisection method
    DISPLAY 6-column iteration table
    PLOT results
END

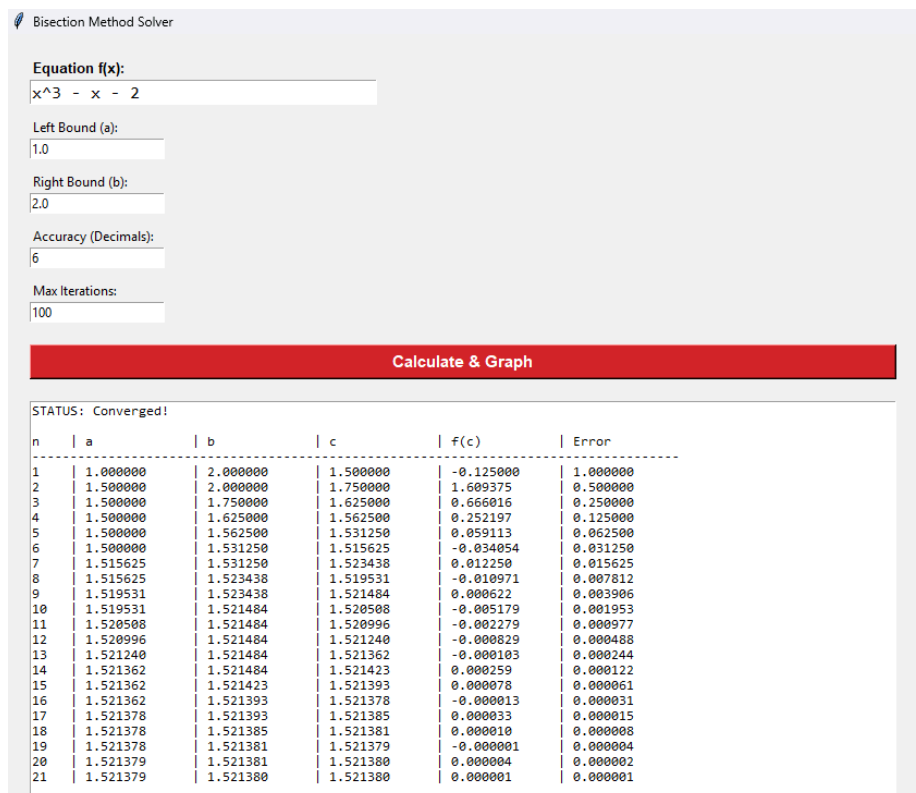
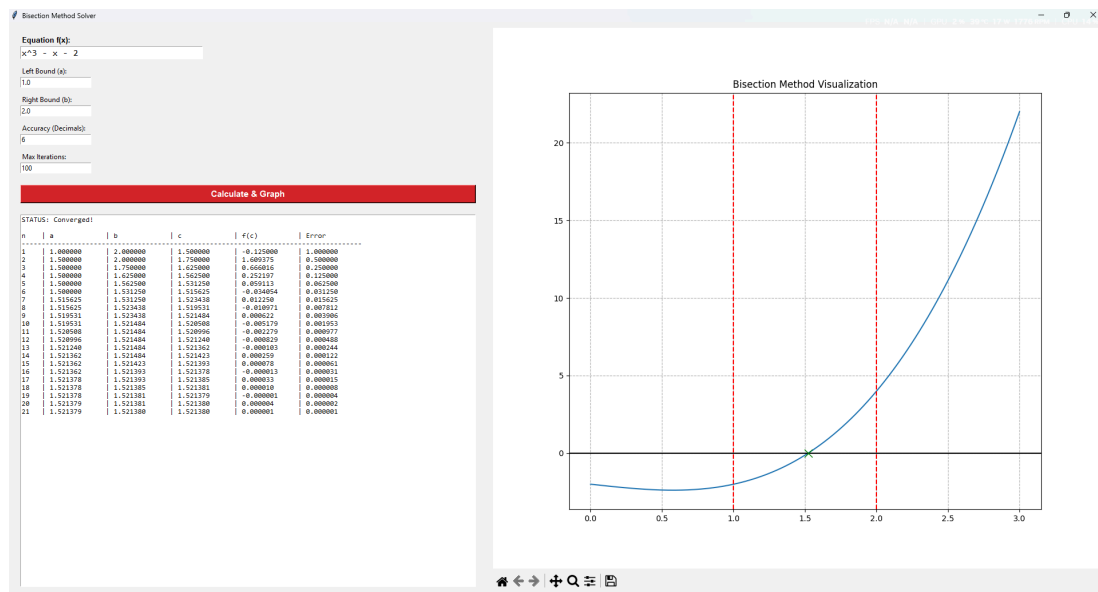
FUNCTION: solve_equation()
START
    READ equation string, a, b, acc, max_iterations
    COMPUTE tolerance = 10^(-acc)
    TRY:
        CALL build_function(equation) → (f)
        CALL bisection_solver(f, a, b, tolerance, max_iterations) → (steps, status)
        DISPLAY status and table header
        FOR each step in steps: DISPLAY formatted values
        CALL plot_graph(f, steps, a, b)
    CATCH error: DISPLAY error message
END

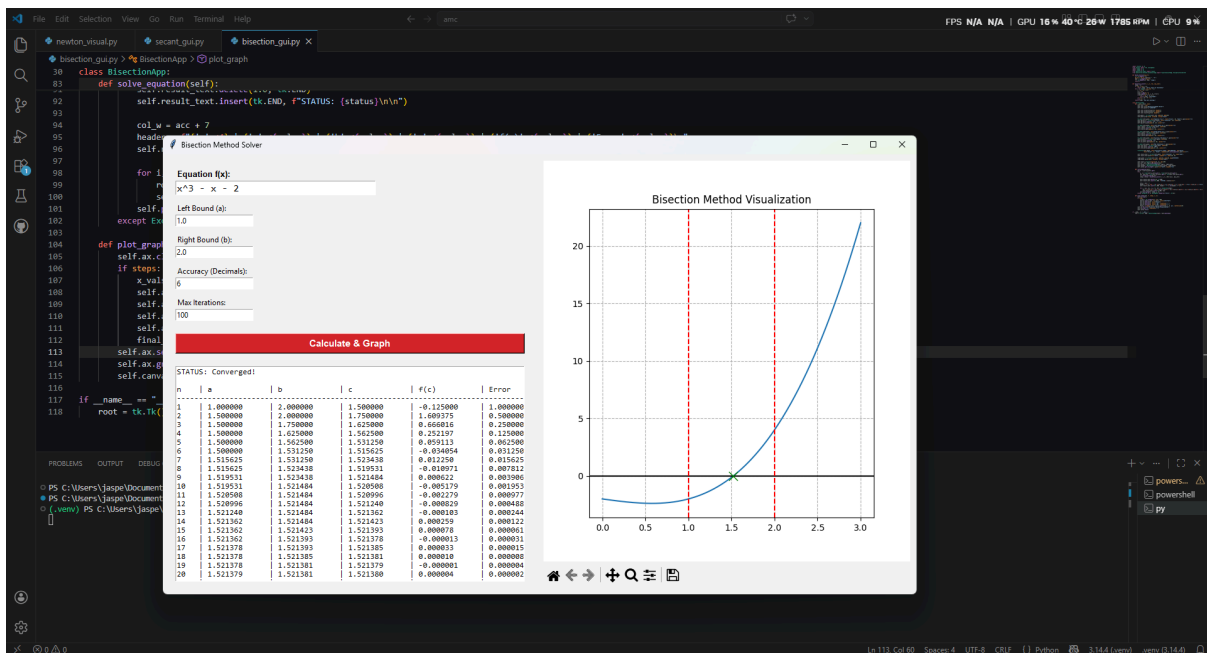
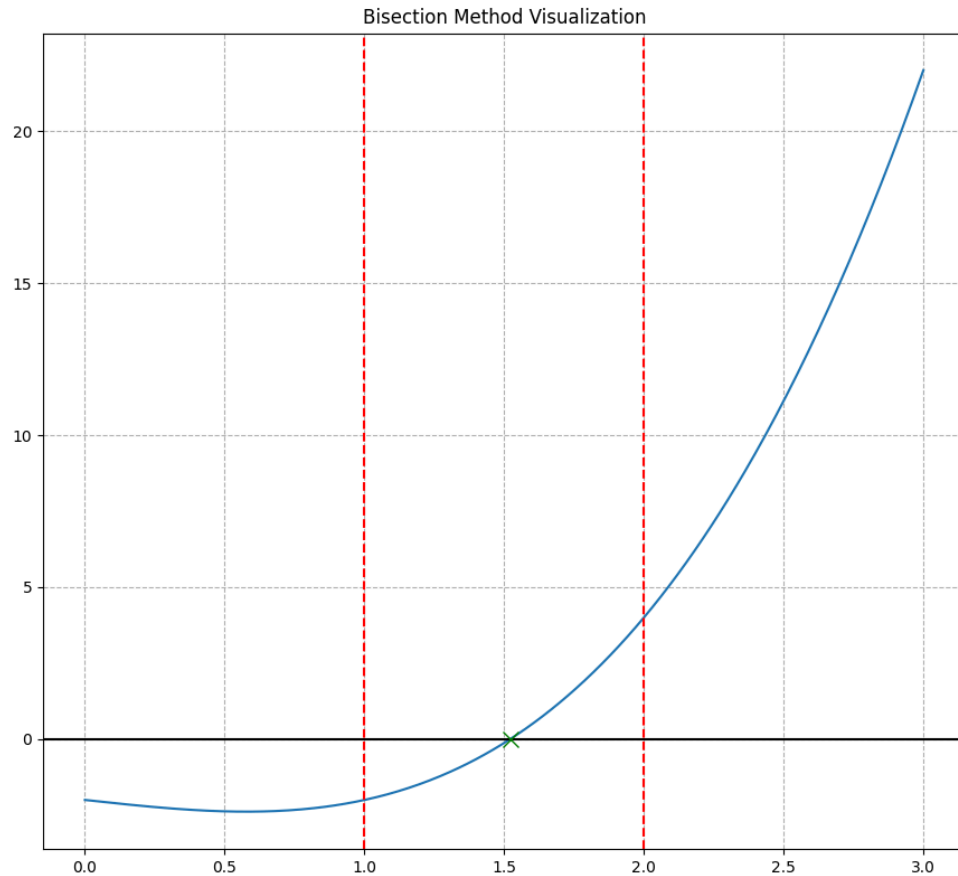
FUNCTION: bisection_solver(f, a, b, tol, max_iter)
    INITIALIZE empty list steps
    IF f(a) * f(b) >= 0: RETURN steps, "Error: Root not bracketed."
    FOR i from 1 to max_iter:
        COMPUTE: c = (a + b) / 2.0
        COMPUTE: fc = f(c)
        COMPUTE: error = |b - a|
        STORE step: (a, b, c, fc, error)
        IF error < tol OR fc == 0: RETURN steps, "Converged!"
        IF f(a) * fc < 0: SET b = c
        ELSE: SET a = c
    RETURN steps, "Did not converge."
```

The Bisection Method is a "bracketing" algorithm based on the Intermediate Value Theorem. It works by "trapping" the root between two points, **a** and **b**, where the function results have opposite signs (one is positive, one is negative). This guarantees that the curve must cross the x-axis somewhere in between.

The program calculates the midpoint c of the interval. It then checks which half of the interval contains the sign change. It throws away the half that doesn't have the root and sets the new boundaries. This effectively cuts the search area in half with every single step. While it is slower than other methods, it is the most reliable because it is mathematically guaranteed to find the root as long as the initial bracket is correct.

III. Test Case & Screenshots





IV. Conclusion

The Bisection Method is a robust and reliable tool for root-finding. Although it requires more iterations to reach high precision compared to Newton or Secant methods, its simplicity makes it an essential algorithm in numerical analysis. This project demonstrates the power of "binary search"

logic applied to mathematics—by consistently halving the search space, we can pinpoint a root to any level of decimal accuracy regardless of the function's complexity.

V. Appendix

```

bisection_gui.py > BisectionApp > plot_graph
1  import tkinter as tk
2  from tkinter import ttk, messagebox
3  import sympy as sp
4  import numpy as np
5  from matplotlib.figure import Figure
6  from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg, NavigationToolbar2Tk
7
8  def build_function(eq_str):
9      x = sp.symbols('x')
10     eq_str = eq_str.replace('^', '**')
11     expr = sp.sympify(eq_str)
12     f = sp.lambdify(x, expr, 'numpy')
13     return f
14
15  def bisection_solver(f, a, b, tol, max_iter):
16      steps = []
17      if f(a) * f(b) >= 0:
18          return steps, "Error: Root not bracketed."
19      for i in range(1, max_iter + 1):
20          c = (a + b) / 2.0
21          fc = f(c)
22          error = abs(b - a)
23          steps.append((a, b, c, fc, error))
24          if error < tol or fc == 0:
25              return steps, "Converged!"
26          if f(a) * fc < 0: b = c
27          else: a = c
28      return steps, "Did not converge."
29
30  class BisectionApp:
31      def __init__(self, root):
32          self.root = root
33          self.root.title("Bisection Method Solver")
34          self.root.geometry("1200x700")
35
36          self.root.columnconfigure(0, weight=1)
37          self.root.columnconfigure(1, weight=2)
38          self.root.rowconfigure(0, weight=1)
39
40          left_panel = tk.Frame(self.root, padx=20, pady=20)
41          left_panel.grid(row=0, column=0, sticky="nsew")
42
43          tk.Label(left_panel, text="Equation f(x):", font=("Arial", 10, "bold")).pack(anchor="w")
44          self.eq_entry = tk.Entry(left_panel, font=("Consolas", 12), width=35)
45          self.eq_entry.insert(0, "x^3 - x - 2")
46          self.eq_entry.pack(pady=(0, 10), anchor="w")
47
48          tk.Label(left_panel, text="Left Bound (a):").pack(anchor="w")
49
50          tk.Label(left_panel, text="Left Bound (a):").pack(anchor="w")
51          self.a_entry = tk.Entry(left_panel, width=20)
52          self.a_entry.insert(0, "1.0")
53          self.a_entry.pack(pady=(0, 10), anchor="w")
54
55          tk.Label(left_panel, text="Right Bound (b):").pack(anchor="w")
56          self.b_entry = tk.Entry(left_panel, width=20)
57          self.b_entry.insert(0, "2.0")
58          self.b_entry.pack(pady=(0, 10), anchor="w")
59
60          tk.Label(left_panel, text="Accuracy (Decimals):").pack(anchor="w")
61          self.acc_entry = tk.Entry(left_panel, width=20)
62          self.acc_entry.insert(0, "6")
63          self.acc_entry.pack(pady=(0, 10), anchor="w")
64
65          tk.Label(left_panel, text="Max Iterations:").pack(anchor="w")
66          self.iter_entry = tk.Entry(left_panel, width=20)
67          self.iter_entry.insert(0, "100")
68          self.iter_entry.pack(pady=(0, 20), anchor="w")
69
70          tk.Button(left_panel, text="Calculate & Graph", bg="#d27228", fg="white",
71                  font=("Arial", 11, "bold"), command=self.solve_equation).pack(fill="x")
72
73          self.result_text = tk.Text(left_panel, font=("Consolas", 9), wrap="none")
74          self.result_text.pack(fill="both", expand=True, pady=(20, 0))
75
76          right_panel = tk.Frame(self.root, padx=10, pady=10, bg="#f0f0f0")
77          right_panel.grid(row=0, column=1, sticky="nsew")
78
79          self.figure = Figure(figsize=(6, 5), dpi=100)
80          self.ax = self.figure.add_subplot(111)
81          self.canvas = FigureCanvasTkAgg(self.figure, right_panel)
82          self.toolbar = NavigationToolbar2Tk(self.canvas, right_panel)
83          self.canvas.get_tk_widget().pack(fill="both", expand=True)
84
85      def solve_equation(self):
86          eq_str = self.eq_entry.get()
87          try:
88              a, b = float(self.a_entry.get()), float(self.b_entry.get())
89              acc, max_iter = int(self.acc_entry.get()), int(self.iter_entry.get())
90              f = build_function(eq_str)
91              steps, status = bisection_solver(f, a, b, 10**(-acc), max_iter)
92
93              self.result_text.delete(1.0, tk.END)
94              self.result_text.insert(tk.END, f"STATUS: {status}\n\n")
95

```

```

85         try:
86             a, b = float(self.a_entry.get()), float(self.b_entry.get())
87             acc, max_iter = int(self.acc_entry.get()), int(self.iter_entry.get())
88             f = build_function(eq_str)
89             steps, status = bisection_solver(f, a, b, 10**(-acc), max_iter)
90
91             self.result_text.delete(1.0, tk.END)
92             self.result_text.insert(tk.END, f"STATUS: {status}\n\n")
93
94             col_w = acc + 7
95             header = f"{'n':<4} | {'a':<{col_w}} | {'b':<{col_w}} | {'c':<{col_w}} | {'f(c)':<{col_w}} | {'Error':<{col_w}}\n"
96             self.result_text.insert(tk.END, header + "-" * len(header) + "\n")
97
98             for i, (a_v, b_v, c_v, fc, err) in enumerate(steps):
99                 row = f"{i+1:<4} | {a_v:<{col_w}.{acc}f} | {b_v:<{col_w}.{acc}f} | {c_v:<{col_w}.{acc}f} | {fc:<{col_w}.{acc}f} | {err:<{col_w}.{acc}f}\n"
100                 self.result_text.insert(tk.END, row)
101             self.plot_graph(f, steps, a, b)
102         except Exception as e: messagebox.showerror("Error", str(e))
103
104     def plot_graph(self, f, steps, a, b):
105         self.ax.clear()
106         if steps:
107             x_vals = np.linspace(a-1, b+1, 400)
108             self.ax.plot(x_vals, f(x_vals), color="#f77b4")
109             self.ax.axhline(0, color="black")
110             self.ax.axvline(a, color='red', linestyle='--')
111             self.ax.axvline(b, color='red', linestyle='--')
112             final_c = steps[-1][2]; self.ax.plot(final_c, 0, 'gx', markersize=10)
113         self.ax.set_title("Bisection Method Visualization")
114         self.ax.grid(True, linestyle='--')
115         self.canvas.draw()
116
117 if __name__ == "__main__":
118     root = tk.Tk(); app = BisectionApp(root); root.mainloop()

```